

Introduction to Distributed Computing

What is Distributed Computing?

Distributed computing is a field of computer science that studies distributed systems, defined as computer systems whose inter-communicating components are located on different networked computers. The components of a distributed system communicate and coordinate their actions by passing messages to one another in order to achieve a common goal.

Distributed Computing ...

It is a collection of independent processes that executes a collection of tasks to coordinate the actions of multiple protocols on a network, such that all components cooperate together to perform a single or small set of related tasks and appears to its users as a single coherent system.

In a distributed computing system, processing and data storage is distributed across multiple devices or systems, rather than being handled by a single central device. Each device or system has its own processing capabilities and may also store and manage its own data. These devices or systems work together to perform tasks and share resources, with no single device serving as the central hub.

Differences between distributed computing and distributed systems

Scope:

- Distributed Computing: Focuses on the algorithms, programming models, and techniques for solving problems in a distributed manner.
- Distributed Systems: Studies the design, implementation, and management of distributed systems, which are computer systems with multiple components located on different networked computers.

Perspective:

- Distributed Computing: Looks at distributed systems from the perspective of solving computational problems efficiently by dividing them into smaller tasks that can be executed concurrently on multiple computers.
- Distributed Systems: Studies the challenges and principles involved in building and maintaining distributed systems, such as dealing with failures, concurrency, and lack of a global clock.

Differences between distributed computing and distributed systems

Emphasis:

- Distributed Computing: Emphasizes the development of distributed algorithms and programming models to harness the power of multiple computers for solving complex problems.
- Distributed Systems: Focuses on the design and implementation of the underlying infrastructure and middleware that enables the coordination and communication between components in a distributed system.

Applications:

- Distributed Computing: Finds applications in areas like parallel processing, grid computing, and cloud computing, where the goal is to leverage the collective computing power of multiple machines.
- Distributed Systems: Has a wider range of applications, including client-server architectures, peer-to-peer networks, and service-oriented architectures, where the focus is on enabling the sharing of resources and services among networked computers.

Why are DCs are gaining popularity

- **Resource and Information Sharing** : The user of computer A may want to use a fancy laser printer connected with computer B, or the user of computer B may need some extra disk space available with computer C for storing a large file.
In a network of workstations, workstation A may want to use the idle computing powers of workstations B and C to enhance the speed of a particular computation.
- **Higher Throughput:** Dividing a total problem into smaller subproblems, and assigning these subproblems to separate physical processors that can operate concurrently is potentially an attractive method of enhancing the speed of computation. Quite often, this is simpler and more economical than investing in a single superfast uniprocessor.

- **Fault Tolerance:** Powerful uniprocessors, or computing systems built around a single central node are prone to a complete collapse when the processor fails. By incorporating redundant processing elements in a distributed system, one can potentially increase system reliability or system availability.
- **Scalability:** An implementation of a distributed system is considered scalable, when its performance can be improved by incrementally adding resources regardless of the final scale of the system.
- **Inherently Distributed Applications:** As an example, consider a banking network. Each bank is supposed to maintain the accounts of its customers. In addition, banks communicate with one another to monitor inter-bank transactions, or record fund transfers from geographically dispersed ATMs.

Examples of Distributed Systems

Google Cloud

Amazon Web Services

Banking networks

Distributed Process control systems

Edge Computing : Smart Cities, Industrial IoT

Distributed machine Learning and AI

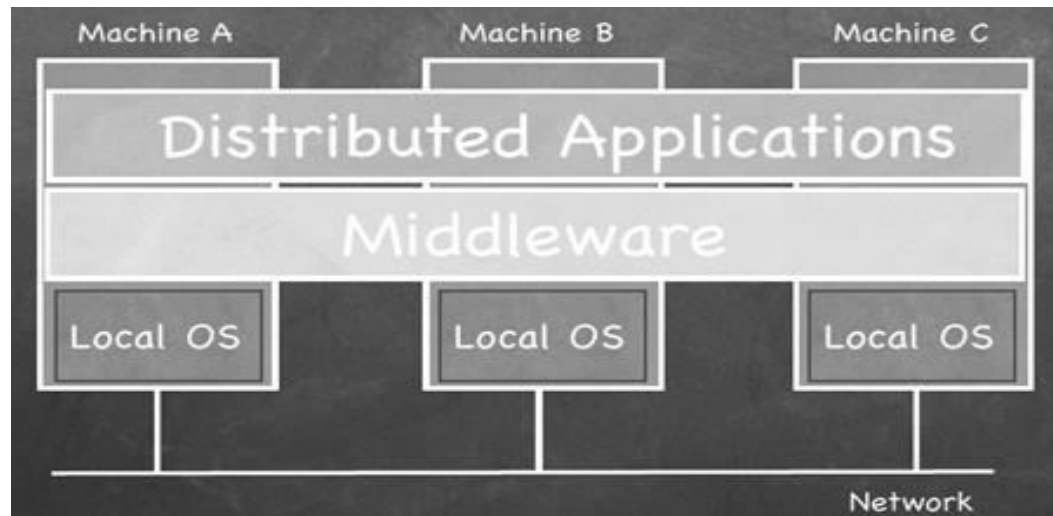
DS must have the following characteristics:

- **Fault-Tolerant:** It can recover from component failures without performing incorrect actions.
- **Highly Available:** It can restore operations, permitting it to resume providing services even when some components have failed.
- **Recoverable:** Failed components can restart themselves and rejoin the system, after the cause of failure has been repaired.
- **Consistent:** The system can coordinate actions by multiple components often in the presence of concurrency and failure. This underlies the ability of a distributed system to act like a non-distributed system.

- **Scalable:** It can operate correctly even as some aspect of the system is scaled to a larger size. For example, we might increase the size of the network similarly, we might increase the number of users or servers. In a scalable system, this should not have a significant effect.
- **Predictable Performance:** The ability to provide desired responsiveness in a timely manner.
- **Secure:** The system authenticates access to data and services

Organization of a distributed system

To support heterogeneous computers and networks while offering a single system view, distributed systems are often organized by means of a layer of software between a higher layer of users and applications and a lower layer of operating systems. Such a distributed system is called a **middleware**.



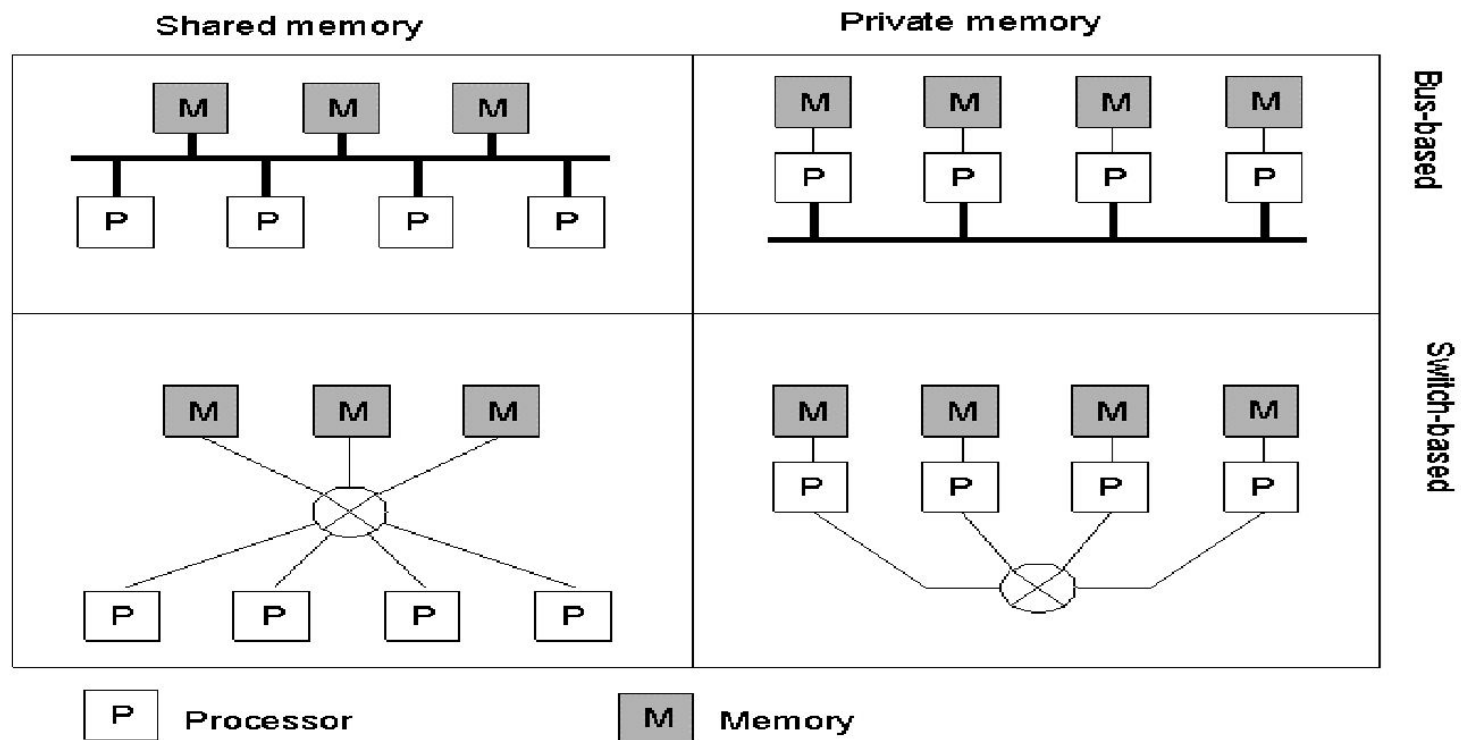
- ***TCS are referred as Paralleling processing System***

Distributed systems can also be implemented on a tightly coupled MIMD machine, where processes running on separate processors are connected to a globally shared memory. In this implementation, the shared memory is used to simulate the interprocess communication channels.

- ***LCS are referred as Distributed system***

Computer architecture: TCS & LCS

- TCS : Single Systemwide primary memory (address space) shared by multiple processors.



Software concepts

- Operating systems for distributed systems have 2 main goals:
 - They act as *resource managers* for the underlying hardware, allowing multiple users and applications to share resources like CPUs, memories, peripheral devices, the network and data of all kinds.
 - They attempt to *hide the intricacies and heterogeneous nature of the underlying hardware* by providing a virtual machine on which the applications can be easily executed.

OS for distributed systems

- *Distributed Operating Systems (DOS)*
 - Tightly-coupled OS (Acting as virtual uniprocessor)
 - Tries to maintain a single, global view of the resources it manages
 - Used for managing multiprocessors and homogeneous multicomputers
 - Dynamically and automatically allocates job to various machines
- *Network Operating Systems (NOS)*
 - Loosely-coupled OS
 - Makes local services available to remote clients
 - Used for managing heterogeneous multicomputer systems
 - To provide better distribution transparency to distributed applications, enhancements to the services of NOS in the form of *middleware* are needed

Goals of a distributed system

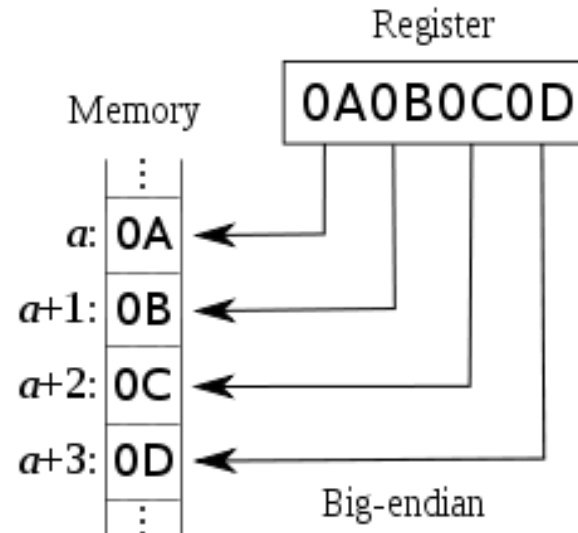
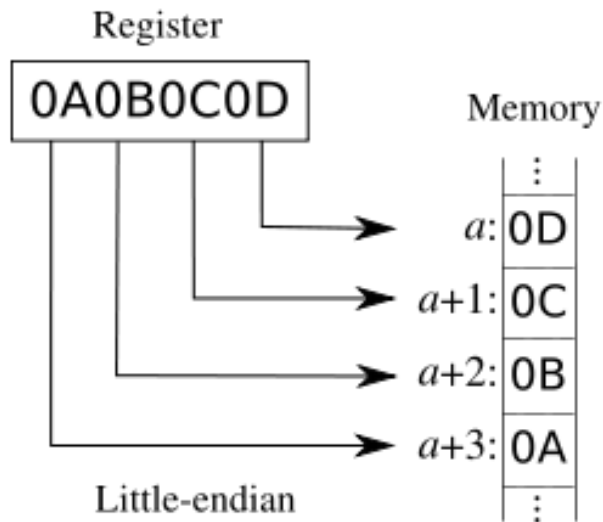
- Connecting users and resources
- Transparency
- Openness
- Scalability

Transparency

- Make the existence of multiple computer invisible and provide a single system image to its users.
- Hide the fact that the resources are physically distributed across multiple computers.
- The eight forms of transparency identified by ISO's "Reference Model for open Distributed Processing".

1. Access transparency

- Distributed os should allow the user to access remote resources in the same way as local resources.
- Hide differences in data representation and how a resource is accessed.



2. Location transparency

- *Name transparency* : The name of a resource should not reveal any hint as to the physical location of the resource . Resources must be able to move from one node to another and thus the names must be unique systemwide.
- *User Mobility*: No matter into which machine a user is logged in he should be able to access a resource with the same name.



3. Replication transparency

- *Resources can be replicated*
 - to increase availability
 - to improve performance by placing a copy close to the place from where it is accessed
- Replication transparency hides the fact that several copies of a resource exist.
- *Generally replication transparency => location transparency*

4. Failure transparency

- Masks from users' partial failure in the system such as link failure, m/c failure, storage device crash etc.
- Issue:
 - It is difficult to achieve since it is hard to distinguish between a dead resource and a painfully slow resource.

5. Migration transparency

- Migration decisions should be made automatically by the system.
- Migration of an object from one node to another should not require any change in name.
- When the migrating object is a process , the inter process communication should ensure that a message sent to the migrating process reaches it without the need for the sender process to resend it if the receiver process moves to another node before the message is received.

6. Concurrency transparency

- i) *Event –Ordering Property*
- ii) *A mutual exclusion property*
- iii) *A no- starvation property*
- iv) *A no-deadlock property*

7. Performance transparency

- Allow the system to automatically reconfigure to improve performance.

8. Scaling transparency

- Allow the system to expand in scale without disrupting the activities of the users.
- Calls for open-system architecture and the use of scalable algorithms for designing DS os components.

Scaling problems [1 of 3]

- *Scaling wrt size:*
 - In a single server and multiple clients, server is overloaded
 - Single server is sometimes unavoidable such as when it stores confidential information.
 - Features of decentralized algorithms:
 - No machine has complete information about the system state
 - Machines make decisions based on local information
 - Failure of one machine does not ruin the algorithm
 - There is no implicit assumption that a global clock exists

Scaling problems [2 of 3]

- **Scaling wrt geography:**
 - Many distributed systems designed for LANs are based on synchronous communication. This leads to unacceptable delay for wide area systems.
 - Local area communication is generally reliable and supports broadcasting. Wide area communication is inherently unreliable and virtually always point-to-point. Hence service query is easier in LAN. Needs special location service in WAN.
 - Centralized services hinder geographical scalability.
 - Example: A central mail server for an entire country.

Scaling problems [3 of 3]

- **Scaling wrt administrative domains:**
 - Conflicting policies regarding resource usage (and payment), management and security.



Guiding Principles for designing scalable DS

- *Avoid Centralized Entities:*
- *Avoid centralized Algorithms: E.g. Centralized scheduling algorithms.*
- *Perform most operations on client workstations*

Design a distributed system with "8 Fallacies"

- The network is reliable.
- Latency is zero.
- Bandwidth is infinite.
- The network is secure.
- Topology doesn't change.
- Transport cost is zero.
- The network is homogeneous.

Issues to handled:

- Ordering events in the absence of a global clock
- Leader election
- Un-reliability of communication
- Capturing the global state with lack of global knowledge
- Mutual Exclusion
- Lack of synchronization and causal ordering
- Managing A large Number of Distributed Resources
- Replica management
- Failure and recovery
- Termination detection

Why is it hard to design them?

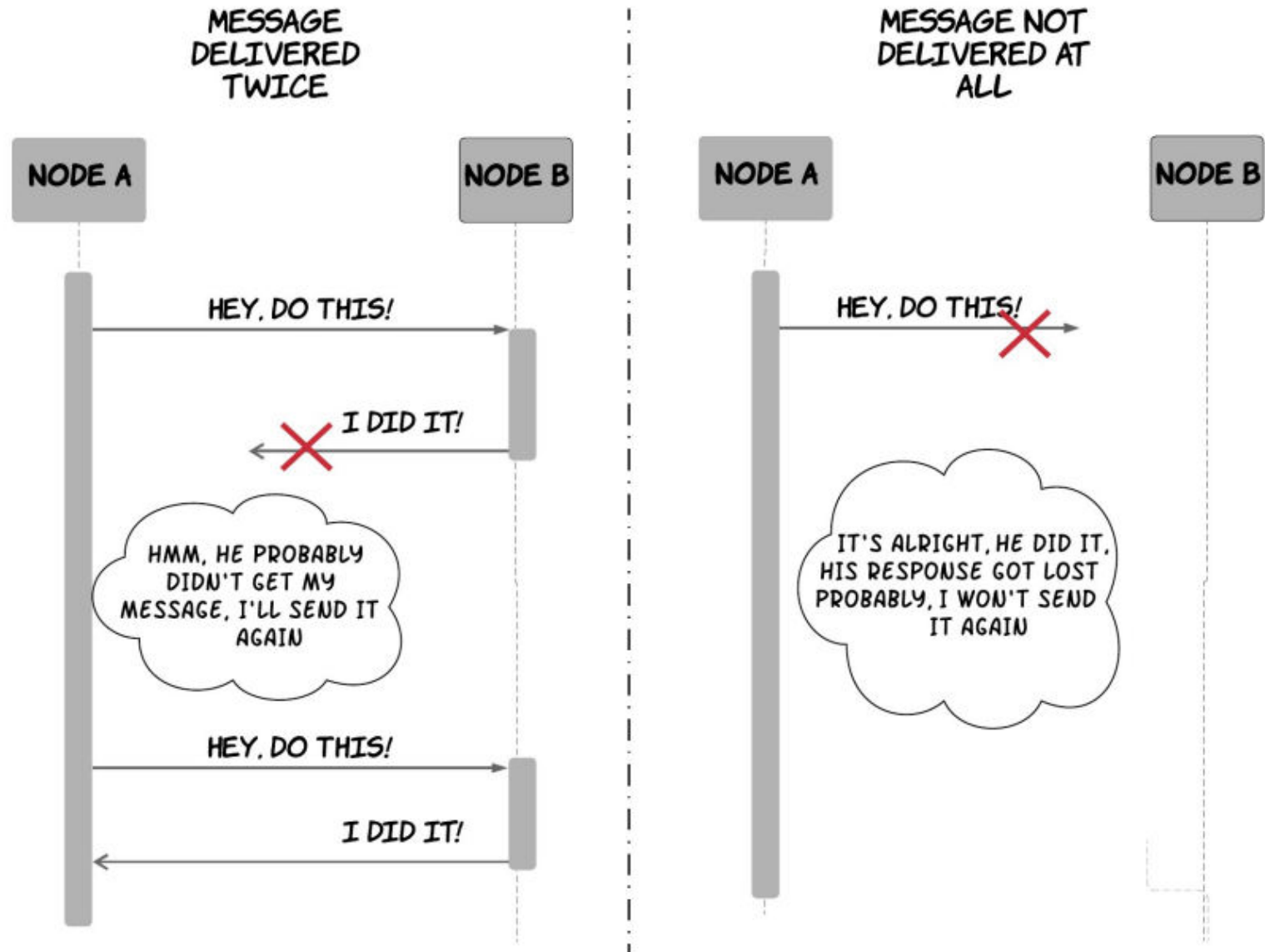
The usual problem of concurrent systems:

- Arbitrary interleaving of actions makes the system hard to verify

+

- No globally shared memory (therefore hard to collect global state)
 - No global clock
 - Unpredictable communication delays

Intricacies of a non-reliable network in distributed systems

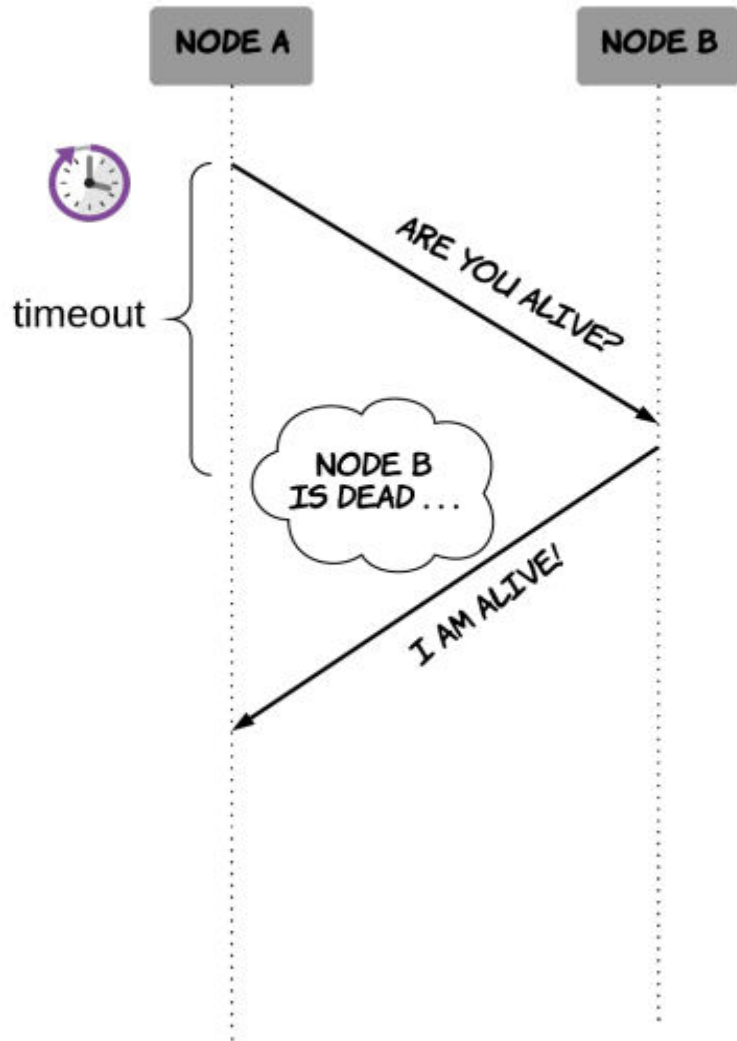


Models for Distributed Algorithms

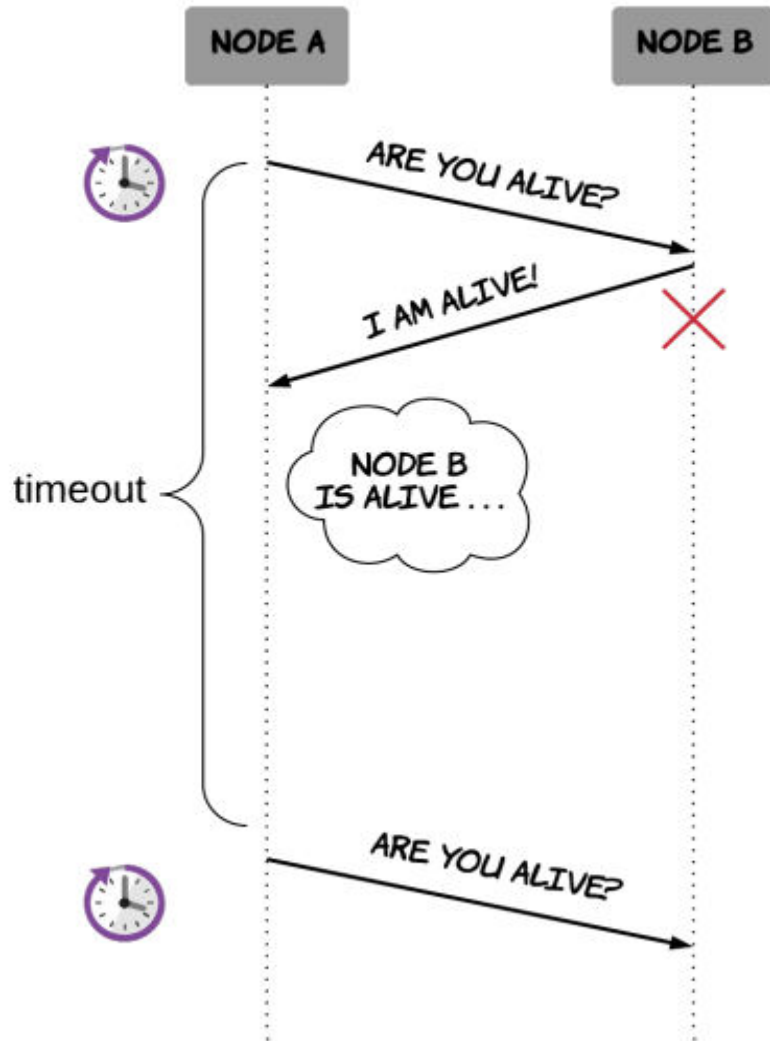
- Topology: Completely connected, Ring, Tree etc.
- Communication : Shared memory / Message passing
(**reliable? Delay? FIFO/Causal? Broadcast/multicast?**)
- Synchronous/Asynchronous
- Failure models: Fail stop, Crash, Omission, Byzantine...
- An algorithm needs to specify the model on which it is supposed to work

Trade-offs in failure detection

FALSE POSITIVES



FALSE NEGATIVES



Complexity Measures

- Message complexity: no. of messages
- Communication complexity / Bit Complexity:
no. of bits
- Time complexity:
 - For synchronous systems, no. of rounds
 - For asynchronous systems, different definitions are there.

Redefining File System Design : GFS

- GFS architecture thrives in an environment where component failures are common, necessitating continuous monitoring and fault tolerance mechanisms for seamless operation.
- Adaptation to handling massive files, in the multi-GB range, with multiple application objects efficiently, leading to a reevaluation of I/O operations and block sizes for enhanced performance.
- Focussing to an data mutation model catering to sequential reads, ensuring atomicity, while aligning with diverse data access patterns.

Design constraints

- Component failures are the norm
 - 1000s of components: inexpensive servers and clients
 - Bugs, human errors, failures of memory, disk, connectors, networking, and power supplies
 - Constant Monitoring, error detection, fault tolerance, automatic recovery are integral to the system
- Files are huge by traditional standards
 - Multi-GB files are common; each file contains application objects such as web documents
 - Billions of objects

Design constraints

- Most modifications are appends
 - Random writes are practically nonexistent
 - Many files are written once, and read sequentially
- Types of reads
 - Data Analysis Programs reading large repositories
 - Large streaming reads (read once)
 - Small random reads (in the forward direction)
- Sustained bandwidth more important than latency
- File system APIs are open to changes

Syllabus...

Fundamental concepts: Models, Issues, Complexity measures, proving correctness

Clocks and Event Ordering: Concept of clock in Distributed System, Limitation of Distributed System, Clock synchronization, Lamport's Logical Clock, Vector Clocks, Causal ordering of messages- Birman-Schiper Stephen Protocol, Schiper Egli Sandoz Protocol.

Global state and snapshot recording algorithms: System model, Snapshot algorithms for FIFO channels, Variations of Chandy-Lamport algorithm, Snapshot algorithms for non-FIFO channels, Snapshots in a causal delivery system, Monitoring global state, Necessary and sufficient conditions for consistent global snapshots, Finding consistent global snapshots in a distributed computation.

Termination detection: Introduction and issues

Fundamental Algorithms: Wave and Traversal Algorithms: Definition and use of wave algorithms, A collection of wave algorithms, Traversal algorithms, Leader Election algorithms: Introduction, Ring Networks, Arbitrary networks, The Korach-Kutten-Moran Algorithm.

Syllabus

Distributed Mutual exclusion: Permission based (Lamport, Ricart-Agarwala, Roucairol-Carvalho, Maekawa), Quorum-based mutual exclusion algorithms (Maekawa's algorithm, Agarwal–El Abbadi quorum-based algorithm), Token based (Suzuki Kasami, Raymond's Tree-based algorithm)

Deadlock detection: System model, Models of deadlocks, Knapp's classification of distributed deadlock detection algorithms, Mitchell and Merritt's algorithm for the single resource model, Chandy–Misra–Haas algorithm for the AND model, Chandy– Misra–Haas algorithm for the OR model, Kshemkalyani–Singhal algorithm for the P-out-of-Q model

Distributed File System: NFS, Google file systems

Distributed Transactions: The Management of Distributed Transactions, A Framework for Transaction Management , Supporting Atomicity of Distributed Transactions, Concurrency Control for Distributed Transactions, Architectural Aspects of Distributed Transactions;

Distributed Concurrency Control: Foundation of Distributed Concurrency Control, Distributed Deadlocks, Concurrency Control based on Timestamps, Optimistic Methods for Distributed Concurrency Control, Execution schedules

Course Outcome

After completing this course the students should be able to:

CO1: Describe and **illustrate** fundamental concept of distributed computing, causality and general framework of logical clocks in distributed environment.

CO2: Sketch and **examine** the concept of Global States and Snapshot Recording Algorithms and extend them to **solve** termination detection.

CO3: Analyze different distributed mutual exclusion algorithms to **solve** simple problems.

CO4: Examine different Deadlock detection algorithms for various types of resource models.

CO5: Be familiar with different Distributed File Systems and **compare** their application in distributed environment.

CO6: Examine different Distributed Database handling issues of transaction processing and Concurrency control.

Reference Books:

1. Advanced Concepts in Operating Systems By Mukesh Singhal and Niranjana G. Shivaratri –McGraw Hill International Edition
2. Introduction to Distributed Algorithms By Gerard Tel – Cambridge University Press
3. Distributed Algorithms By Nancy A Lynch
4. Distributed Operating Systems Concepts and Design By George Coulouris, Jean Dollimore, Tim Kindberg – Pearson Education